

## Quiz 4 Problem Set

Quiz 4 will be based on these parts of the textbook:

- Chapter 11: 11.1, 11.2, 11.3
- Chapter 10: 10.1, 10.2

The questions on the quiz may refer to definitions, theorems, and examples from the textbook. The questions could be of any type, and will be variations of questions below, or questions that are similar.

**Important:** *Treat these problem-sets as non-AI activities!* Turn off all AI support and try to figure them out yourself **by hand**. Having AI or another student do this for you will not help you learn. You must do the learning yourself!

### Question 1

Give detailed C++-like pseudocode implementations of:

- a) Merging two already-sorted lists into a new list.
- b) Mergesort.

### Question 2

Give a detailed mathematical explanation of **Proposition 11.2** (that mergesort sorts in worst-case  $O(n \log n)$  time).

### Question 3

Give a detailed C++-like pseudocode implementation of:

- a) Partitioning the elements of an array, using its first element as a pivot.
- b) Quicksort.

### Question 4

- a) What is the average-case running time of quicksort? Justify your answer with a mathematical explanation.
- b) What is the worst-case running time of quicksort? Give an example of an unsorted array with 100 elements that results in a worst-case performance.
- c) Explain how randomized quicksort works. What is its expected running time?

## Question 5

- a) Give a mathematical explanation for why a general-purpose comparison-based sorting routine must have a worst case of at least  $O(n \log n)$  (Proposition 11.4).
- b) Using the result from a), show that it's **impossible** to implement a Priority Queue ADT (from chapter 8) such that both `insert` and `removeMin` have worst-case  $O(1)$  running time.

## Question 6

For the following algorithms, give C++-like pseudocode for them, and give and justify their worst-case running time:

- a) Bucket sort
- b) Radix sort

## Question 7

- a) Give the definition of a **binary search tree (BST)**.
- b) Are all **min heaps** also BSTs? If so, explain why. If not, give an example of a heap that is **not** a BST.
- c) Do all BSTs satisfy the **heap-order property** (for min heaps)? If so, explain why. If not, give an example of a BST that does **not** satisfy the heap-order property.

## Question 8

Give detailed C++-like pseudocode that answers these questions:

- a) How many nodes are in a BST?
- b) How many leaf nodes does a BST have?
- c) What is the sum of the values in a BST (assume all values are ints)?
- d) Search for a value  $x$  in the tree. If it's in the tree, return `true`, otherwise return `false`.

## Question 9

- a) State the **height-balance property** for a binary tree.
- b) Define an **AVL tree**.
- c) In the worst case, what is the **height** of an AVL tree with  $n$  nodes?
- d) When inserting a new value  $x$  into an AVL tree, draw diagrams of **the four trinode restructuring operations** that may be needed to ensure the tree remains height-balanced.

## Question 10

Starting with an initially empty tree, insert the following values, in the order given, into an AVL tree: **1, 2, 3, 4, 8, 7, 6, 5**

Draw the tree after each insertion.